OSM Marking Screen — Deep-Dive Knowledge Transfer

> **Scope:** This document covers only the live OSM script marking interface — the pages, canvas components, and toolbar that a faculty evaluator actually uses to mark a scanned exam script. Legacy/unused files (PBT, GradingPage component, react-sketch-canvas, AnnotationContextMenu) are called out explicitly so you know to ignore them.

Table of Contents

---

1. What the Marking Screen Is

When an evaluator clicks on a script assignment in their dashboard, they are taken to:

```
/osm/grading/:uniqueScriptCode?rosterId=<id>&flow=<osm|nu>
```

This route renders `OSMGradingPage`. It is a full-page, fullscreen-capable evaluation workspace with three panes:

If `metaData.bluePrintDetails.blueprint_url` is present, a **View Blueprint** button appears in the header. Clicking it opens `DraggablePdfViewer` — a floating, draggable, resizable PDF window for the question paper blueprint, so the evaluator can refer to it without leaving the screen.

If the evaluator exits fullscreen (ESC, F11, etc.), a fullscreen-prompt overlay re-appears asking them to re-enter.

---

2. File Map — What's Active vs Unused

Active Files (what this document covers)

File	Role
`src/pages/OSM/OSMGradingPage.jsx`	The page component — orchestrates everything
`src/pages/OSM/GradingToolbar.jsx`	Left sidebar toolbar (tool selection, actions)
`src/components/Canvas/PDFCanvas.jsx`	Core canvas: renders PDF + Konva drawing layer
`src/components/Canvas/DraggableColorPalette.jsx`	Floating draggable colour picker
`src/components/Canvas/NumberedCommentsSidebar.jsx`	Right panel section for numbered comments list
`src/components/Canvas/DraggablePdfViewer.jsx`	Floating draggable blueprint viewer

Unused / Legacy Files (do NOT use)

File	Why Unused
`src/components/Canvas/AnnotationContextMenu.jsx`	Built for right-click context menu on annotations. Never wired into `PDFCanvas` — no `onContextMenu` handler exists. Do not use.
`src/components/GradingPage/GradingPage.jsx`	Old generic grading component. Replaced by `OSMGradingPage`. Still registered at `/grading` route but not part of the OSM flow.
`src/components/GradingPage/CBTLayout.jsx`	Only used inside old `GradingPage`. Unused in OSM.
`src/components/GradingPage/PBTLayout.jsx`	PBT (Paper-Based Test) layout. Not part of any active flow.
`src/components/GradingPage/MarksPanel.jsx`	Old marks panel, superseded by the inline scoring panel in `OSMGradingPage`.
`src/components/GradingPage/QuestionsPanel.jsx`	Old questions panel, not used in OSM.
`src/components/GradingPage/QuestionDetails.jsx`	Old question details panel, not used.
`src/components/PBT/PBTMarksPanel.jsx`	PBT-specific, not in OSM flow.
`src/components/PBT/PBTQuestionsPanel.jsx`	PBT-specific, not in OSM flow.
`src/components/PBT/PBTQuestionDetails.jsx`	PBT-specific, not in OSM flow.
`react-sketch-canvas` (package)	Installed but never imported in any active file. Was an earlier attempt at annotation; fully replaced by `react-konva`.

---

3. Component Hierarchy

```
OSMGradingPage
│
├── [overlay] Fullscreen Prompt Banner
│
├── <FullScreen handle={fullScreenHandle}>     ← react-full-screen wrapper
│   │
│   ├── <Toaster />                            ← react-hot-toast (inside fullscreen so toasts show above fullscreen layer)
│   │
│   ├── <StyledBreadcrumb />                   ← navigation breadcrumb
│   │
│   ├── <header>                               ← page header bar
│   │   ├── Script code title + page input
│   │   ├── [conditional] View Blueprint button
│   │   ├── [conditional] Exit Fullscreen button
│   │   ├── Save Progress button
│   │   └── Submit Final Evaluation button
│   │
│   ├── <main layout div>
│   │   ├── [conditional] <DraggableColorPalette />    ← floats over everything when open
│   │   │
│   │   ├── <GradingToolbar />                         ← left pane (140px wide)
│   │   │
│   │   ├── <PDFCanvas ref={canvasRef} />              ← centre pane (flex-1)
│   │   │   ├── [overlay] comment text input popup
│   │   │   ├── <Document> (react-pdf)
│   │   │   │   └── For each page:
│   │   │   │       ├── <Page /> (react-pdf)          ← renders PDF page as image
│   │   │   │       └── <Stage> (react-konva)         ← drawing canvas overlay
│   │   │   │           └── <Layer>
│   │   │   │               ├── <Rect fill="transparent" />   ← click/drag catchall
│   │   │   │               ├── Annotations (Line, Text, Rect, Label)
│   │   │   │               └── [conditional] tempCommentRect while dragging
│   │   │   └── [overlay] "Page X / N" indicator
│   │   │
│   │   └── <aside> (right pane — 384px wide)
│   │       ├── "Manual Scoring" header
│   │       ├── Score rows (question number + marks input)
│   │       ├── "Add Score Entry" button (only if no marking pallet)
│   │       ├── <NumberedCommentsSidebar />
│   │       ├── Total marks display
│   │       └── Overall remarks textarea
│   │
│   └── [conditional] <DraggablePdfViewer />    ← blueprint viewer, floats over everything
```

---

4. Libraries Used (Full Detail)

4.1 `react-pdf` v9.2.1

What it does: Renders PDF files in the browser using Mozilla's `pdf.js` engine under the hood.

Key concepts:

- `<Document>` — top-level component. Takes a `file` prop which can be a `Blob`, URL string, `ArrayBuffer`, or base64 string.
- `<Page>` — renders a single page of the PDF. Takes `pageNumber` (1-indexed) and `scale` props.
- `pdfjs.GlobalWorkerOptions.workerSrc` — must point to a web worker script. In `PDFCanvas.jsx`, it uses the unpkg CDN version; in `DraggablePdfViewer.jsx`, it uses a local bundled worker via `import.meta.url`. Both approaches are valid.

Props used in this codebase:

```
// PDFCanvas.jsx
<Document
  file={pdfBlob}                  // a Blob object (downloaded from SAS URL)
  onLoadSuccess={({ numPages })}  // fires once PDF is parsed
  className="flex flex-col items-center gap-4 p-4"
>
  <Page
    pageNumber={index + 1}
    scale={scale}                  // zoom factor (zoom / 100)
    renderTextLayer={false}        // disables selectable text overlay (we don't need it)
    renderAnnotationLayer={false}  // disables PDF's own annotations (we draw our own)
    onRenderSuccess={onPageRenderSuccess}  // gives us page viewport dimensions
  />
</Document>

// DraggablePdfViewer.jsx
<Document file={pdfUrl}>          // a URL string (Azure SAS URL)
  <Page
    pageNumber={currentPage}
    renderTextLayer={false}
    renderAnnotationLayer={false}
    width={380 * scale}            // fixed-width approach (not scale prop)
  />
</Document>
```

`onPageRenderSuccess` callback: This is called after the first page renders. We extract the viewport dimensions at `scale: 1.0` to get the **base dimensions** of the PDF page. These are critical for the annotation normalisation system (see Section 13).

Why the PDF is fetched as a Blob: The script PDF is behind an Azure SAS URL. Rather than passing the URL directly to `<Document>`, `OSMGradingPage` fetches it via the Fetch API and stores the `Blob`. This avoids CORS issues and gives us more control over loading state. The `Blob` is stored in component state (`pdfBlob`).

4.2 `react-konva` v18.2.10 + `konva` v9.3.14

What it does: A React wrapper around the Konva.js canvas library. Konva renders 2D shapes onto an HTML5 `<canvas>` element and provides a scene-graph abstraction (similar to SVG's DOM).

Why Konva instead of raw canvas: Konva gives us:

- Per-shape event handling (`onClick`, `onMouseDown`, etc.)
- Hit detection (know exactly which shape was clicked)
- Layer management (separation between static and dynamic content)
- Easy serialisation of shape properties

Key Konva components used:

Component	Konva equivalent	Purpose in this app
`<Stage>`	Konva.Stage	The root canvas element. Set to exact PDF page dimensions.
`<Layer>`	Konva.Layer	A canvas layer. All annotations are in one layer.
`<Line>`	Konva.Line	Pen strokes and highlighter strokes. Uses `tension=0.5` for smooth curves, `lineCap="round"`, `lineJoin="round"`.
`<Text>`	Konva.Text	Tick (✓), Cross (✕), Star (★) stamps and numbered comment text.
`<Rect>`	Konva.Rect	Transparent hit area over the whole page, numbered comment boxes, selection highlight boxes, temp drag rectangle during comment creation.
`<Label>`	Konva.Label	Container for `<Tag>` + `<Text>` — used to render the purple badge number on numbered comments.
`<Tag>`	Konva.Tag	Filled background for the label badge (purple pill).

How Stage and Page are aligned:

```
<div className="relative">     ← PDF page wrapper div
  <Page />                     ← react-pdf renders the PDF as a raster image
  <Stage                       ← Konva canvas sits exactly on top
    width={basePageDimensions.width * scale}
    height={basePageDimensions.height * scale}
    className="absolute top-0 left-0"   ← overlays perfectly
  >
```

The `Stage` and `Page` dimensions are always synchronised via the `scale` value derived from the zoom state.

`globalCompositeOperation` for highlighter:

```
<Line
  globalCompositeOperation={
    item.type === "highlighter" ? "multiply" : "source-over"
  }
/>
```

- `"source-over"` (default) — normal drawing: new strokes appear on top
- `"multiply"` — blends colours, making the highlighter transparent-like. Yellow on white = yellow; yellow on text = slightly darkened, looks like a real highlighter.

`tension` on Line: `tension={0.5}` makes Konva draw a smooth Catmull-Rom spline through the points instead of sharp line segments. This gives a natural pen stroke appearance.

Event handling on Layer:

```
<Layer
  onMouseDown={(e) => handleMouseDown(e, index + 1)}
  onMouseMove={(e) => handleMouseMove(e, index + 1)}
  onMouseUp={(e) => handleMouseUp(e, index + 1)}
  onMouseLeave={(e) => handleMouseUp(e, 0)}     // finish stroke if cursor leaves
  onTouchStart={(e) => handleMouseDown(e, index + 1)}
  onTouchMove={(e) => handleMouseMove(e, index + 1)}
  onTouchEnd={(e) => handleMouseUp(e, index + 1)}
>
```

All mouse and touch events are handled at the `Layer` level, not on individual shapes. The `pageNumber` is passed as a closure argument so handlers know which page they're operating on.

4.3 `react-draggable` v4.4.6

What it does: Makes any React component draggable via mouse/touch events, without any CSS transforms happening in the DOM — it uses `transform: translate(x, y)` internally.

Props used:

```
<Draggable
  handle=".drag-handle"   // only this CSS class acts as the drag handle
  nodeRef={nodeRef}        // ref to the DOM node (required to avoid deprecated findDOMNode)
  bounds="parent"          // cannot be dragged outside parent element
  cancel="button"          // clicking buttons inside doesn't start a drag
>
  <div ref={nodeRef}>...</div>
</Draggable>
```

Used in:

- `DraggableColorPalette` — the floating colour picker
- `DraggablePdfViewer` — the floating blueprint viewer

Important: The draggable element must have `position: absolute` or `position: fixed` to actually float. Both components use `absolute`.

4.4 `react-colorful` v5.6.1

What it does: A tiny (2.8kB), zero-dependency colour picker component. Renders a hue-saturation square + hue slider.

Component used:

```
import { HexColorPicker } from "react-colorful";

<HexColorPicker
  color={initialColor}   // current hex colour string, e.g., "#D32F2F"
  onChange={onChange}    // called with new hex string on every change (real-time)
/>
```

Only used in: `DraggableColorPalette.jsx`. The picked colour flows up via `onChange` to `OSMGradingPage`, which stores it in `currentColor` or `highlighterColor` state depending on the active tool.

4.5 `react-full-screen` v1.1.1

What it does: A React wrapper around the browser's Fullscreen API. Handles cross-browser prefixes (`webkit`, `moz`, `ms`).

Usage:

```
import { FullScreen, useFullScreenHandle } from "react-full-screen";

const fullScreenHandle = useFullScreenHandle();

// Enter fullscreen programmatically:
fullScreenHandle.enter();

// Exit:
fullScreenHandle.exit();

// Wrap content in:
<FullScreen handle={fullScreenHandle} className="overflow-auto relative">
  {/* content that goes fullscreen */}
</FullScreen>
```

Why `<Toaster>` is inside `<FullScreen>`: When the browser enters fullscreen mode, any elements outside the fullscreen element are hidden. Toast notifications must therefore be rendered inside the `<FullScreen>` wrapper, with a very high `zIndex`, to remain visible during marking.

Fullscreen change detection: `OSMGradingPage` manually listens to browser fullscreen-change events to detect when the user exits fullscreen via ESC or F11:

```
document.addEventListener("fullscreenchange", handleFullScreenChange);
document.addEventListener("webkitfullscreenchange", handleFullScreenChange);
document.addEventListener("mozfullscreenchange", handleFullScreenChange);
document.addEventListener("MSFullscreenChange", handleFullScreenChange);
```

When detected, `showFullscreenBanner` is set to `true` and the overlay prompt re-appears.

4.6 `@tanstack/react-query` v5.59.15

What it does: Server state management and data-fetching library. Handles caching, background refetching, loading/error states.

Used in `OSMGradingPage` for:

```
// 1. Fetching grading metadata (script details, marking pallet, saved progress)
const { data: metaData, isLoading, isError } = useQuery({
  queryKey: ["gradingData", uniqueScriptCode, rosterId, isNuFlow],
  queryFn: () => fetchApi(uniqueScriptCode, user._id, rosterId),
  enabled: !!user?._id && !!rosterId,   // don't fetch until these are ready
});
```

```
// 2. Saving/submitting marks
const saveMutation = useMutation({
  mutationFn: saveApi,
  onSuccess: (response, variables) => {
    toast.success(...);
    queryClient.invalidateQueries({ queryKey: ["gradingData", ...] });
    if (variables.status === "completed") navigate(-1);
  },
  onError: (err) => toast.error(...),
});
```

Key pattern: `saveMutation.mutate(payload)` is called for both "Save Progress" (`status: "in_progress"`) and "Submit Final Evaluation" (`status: "completed"`). The `onSuccess` handler checks `variables.status` to decide whether to navigate away.

`queryClient.invalidateQueries(...)` after save ensures that if the evaluator somehow returns to the same script, they see fresh data (not cached old progress).

4.7 `react-hot-toast` v2.5.2

What it does: Lightweight toast notification system.

Usage in this screen:

```
import toast, { Toaster } from "react-hot-toast";

// Render the container (inside <FullScreen> so it works in fullscreen):
<Toaster containerStyle={{ zIndex: 99999 }} />

// Show notifications:
toast.success("Evaluation submitted successfully!");
toast.error("Marks cannot exceed 15 for this question.");
toast.error("Please enter marks for all questions.");
```

Custom `zIndex` : Without `containerStyle={{ zIndex: 99999 }}` , toasts would appear behind the fullscreen overlay. The `containerStyle` prop passes styles directly to the toast container div.

4.8 `axios` v1.7.7 (via shared `api.js`)

Direct `api.post(...)` calls are used for fetching and saving grading data — not wrapped in React Query service files, since these are defined as local functions inside `OSMGradingPage.jsx` itself:

```
const fetchGradingDataAPI = async (uniqueScriptCode, evaluatorId, rosterId) => {
  const { data } = await api.post(
    `/api/osm/evaluation/script/fetch-grading-data/${uniqueScriptCode}`,
    { rosterId, evaluatorId }
  );
  return data;
};

const saveProgressAPI = async (progressData) => {
  const { data } = await api.post(
    "/api/osm/evaluation/script/save-progress",
    progressData
  );
  return data;
};
```

The `api` instance (from `src/api.js`) auto-attaches the Bearer token, so no manual auth header is needed here.

4.9 `react-router-dom` v6.26.1

Used in `OSMGradingPage` :

```
const { uniqueScriptCode } = useParams();      // from URL: /osm/grading/:uniqueScriptCode
const location = useLocation();                // to read query params
const navigate = useNavigate();                // to navigate back after submission

// Reading query params:
const queryParams = useMemo(() => new URLSearchParams(location.search), [location.search]);
const isNuFlow = queryParams.get("flow") === "nu";
const rosterId = new URLSearchParams(location.search).get("rosterId");
```

4.10 `lucide-react` v0.428.0

Icon library used throughout the toolbar and header. All icons are SVG-based React components. Icons used in this screen:

Icon	Where
`PlusCircle`	"Add Score Entry" button
`Trash2`	Remove score row / delete comment
`FileText`	Page indicator
`Minimize2`	Exit Fullscreen button
`LayoutDashboard`	Breadcrumb icon
`BookOpen`	Blueprint button icon
`MousePointer2`	Select tool in toolbar
`Pen`	Pen tool
`Highlighter`	Highlighter tool
`Eraser`	Eraser tool
`Check`	Tick stamp
`X`	Cross stamp
`Star`	Star stamp
`Hash`	Numbered comment tool
`Palette`	Color palette button
`Minus` / `Plus`	Zoom out / in
`Undo` / `Redo`	History actions
`ChevronLeft` / `ChevronRight`	Blueprint viewer pagination
`ZoomIn` / `ZoomOut`	Blueprint viewer zoom
`Maximize2`	Blueprint minimize/maximize

4.11 `react-redux` + `@reduxjs/toolkit`

Only auth state is read from Redux in this screen:

```
const { user } = useSelector((state) => state.auth);
// user._id is used as evaluatorId in API calls
```

Note: Annotations are **not** stored in Redux here — they live in `PDFCanvas`'s local state. The `annotationsSlice` in Redux was built for a different grading flow and is not used by `OSMGradingPage`.

4.12 `IntersectionObserver` (Web API — not a library)

`PDFCanvas` uses the browser's native `IntersectionObserver` API (no library needed) to detect which PDF page is currently most visible in the scroll container:

```
const observer = new IntersectionObserver(observerCallback, {
  root: scrollContainerRef.current,  // observe within the scroll container
  threshold: 0.1,                    // fire when 10% of element is visible
});
// Observes each .pdf-page-wrapper div
```

When a page crosses the 10% visibility threshold, `setCurrentPage` is called, updating the page counter in the header.

---

5. OSMGradingPage — Complete Breakdown

File: `src/pages/OSM/OSMGradingPage.jsx`
Route: `/osm/grading/:uniqueScriptCode?rosterId=<id>&flow=<osm|nu>`

5.1 URL Parameters and Query Strings

Source	Key	Value	Purpose
URL param	`:uniqueScriptCode`	e.g., `SCR-ABC-123`	Identifies the specific script
Query string	`rosterId`	MongoDB ObjectId	Links to the roster this assignment belongs to
Query string	`flow`	`"osm"` or `"nu"`	Selects which API endpoints to use

5.2 Dual Flow (OSM vs NU)

The page has two distinct API pairs, selected by the `flow` query parameter:

```
const isNuFlow = queryParams.get("flow") === "nu";
const fetchApi = isNuFlow ? fetchNuGradingDataAPI : fetchGradingDataAPI;
const saveApi  = isNuFlow ? saveNuProgressAPI      : saveProgressAPI;
```

Flow	Fetch endpoint	Save endpoint	Data source
`osm`	`/api/osm/evaluation/script/fetch-grading-data/:code`	`/api/osm/evaluation/script/save-progress`	Standard OSM collections
`nu`	`/api/cbt/offline/evaluation/script/fetch-grading-data/:code`	`/api/cbt/offline/evaluation/script/save-progress`	NU-specific (non-university) offline CBT collections

Both API functions take the same signature `(uniqueScriptCode, evaluatorId, rosterId)`. From the component's perspective, the flow is transparent after the function references are assigned.

5.3 State Variables

State	Type	Initial Value	Purpose
`scores`	`ScoreRow[]`	`[]`	The marks breakdown array. Populated from marking pallet or saved progress.
`overallComment`	`string`	`""`	Evaluator's overall remarks.
			Text of numbered comments placed on canvas. Synced with canvas

State	Type	Initial Value	Purpose
`numberedComments`	`CommentObject[]`	`[]`	annotations.
`numPages`	`number|null`	`null`	Total pages in the PDF. Set by PDFCanvas.
`currentPage`	`number`	`1`	Currently visible page. Updated by PDFCanvas via IntersectionObserver.
`zoom`	`number`	`100`	Zoom percentage (50–200). Passed to PDFCanvas.
`selectedTool`	`string`	`"pen"`	Active tool ID.
`selectedCommentId`	`any`	`null`	ID of the currently selected numbered comment (for synced highlighting).
`isPaletteOpen`	`boolean`	`false`	Whether the colour palette floater is visible.
`currentColor`	`string`	`"#D32F2F"`	Active pen colour (hex).
`highlighterColor`	`string`	`"#FFEB3B"`	Active highlighter colour (hex).
`brushSize`	`number`	`5`	Pen stroke width in pixels (1–20).
`pdfBlob`	`Blob|null`	`null`	The downloaded PDF file as a Blob.
`isPdfLoading`	`boolean`	`true`	Whether the PDF Blob is still downloading.
`pdfError`	`string|null`	`null`	Error message if PDF download fails.
`pageInput`	`string`	`"1"`	Controlled value of the page-jump input field.
`showFullscreenBanner`	`boolean`	`true`	Whether the fullscreen prompt overlay is shown.
`isBlueprintVisible`	`boolean`	`false`	Whether the DraggablePdfViewer is open.

5.4 ScoreRow Shape

Each entry in the `scores` array:

```
{
  id: number,             // Date.now() — used as React key; stripped before saving
  questionNumber: string, // e.g., "Q1" or "1a"
  marksAwarded: string,   // number as string, or "" if not yet entered
  maxMarks: number,       // from marking pallet (undefined if no pallet)
  label: string,          // question label from pallet (undefined if no pallet)
}
```

5.5 Score Hydration Logic

When `metaData` arrives from the server, `scores` is populated via a `useEffect`:

```
Priority 1: marking pallet exists?
  → Build rows from metaData.markingPallet.questions
  → For each question, check if metaData.savedProgress.marksBreakdown
    has a saved value → pre-fill if found

Priority 2: no pallet, but saved progress exists?
  → Build rows directly from metaData.savedProgress.marksBreakdown
  → Assign temporary IDs (Date.now() + index)

Priority 3: nothing?
  → scores stays []
  → Evaluator must click "+ Add Score Entry" manually
```

Consequence: If a marking pallet is present, the `questionNumber` fields are **read-only** (rendered with `readOnly` and `tabIndex={-1}`). The evaluator can only fill in the marks. The "Delete row" button is also hidden. This prevents the evaluator from modifying the question structure.

5.6 Score Validation (`handleScoreChange`)

Two validation rules are enforced in real-time as the evaluator types:

1. **Max marks:** If `score.maxMarks` is set (from pallet) and the entered value exceeds it:

```
if (numValue > score.maxMarks) {
  toast.error(`Marks cannot exceed ${score.maxMarks} for this question.`);
  return score; // reject update, keep old value
}
```

2. **Negative marking:** If value is negative AND `metaData.markingPallet.allowNegativeMarking` is false:

```
if (numValue < 0 && !metaData?.markingPallet?.allowNegativeMarking) {
  toast.error("Negative marking is not allowed for this evaluation.");
  return score;
}
```

The `onWheel` handler on mark inputs calls `e.target.blur()` to prevent accidental scroll-to-change on number inputs.

5.7 Keyboard Navigation Between Mark Inputs

The evaluator can navigate between marks inputs using `Ctrl+ArrowUp` and `Ctrl+ArrowDown`:

```
const handleKeyDown = (e, index) => {
  if (e.ctrlKey) {
    if (e.key === "ArrowUp") {
      document.getElementById(`marks-input-${index - 1}`)?.focus();
    } else if (e.key === "ArrowDown") {
      document.getElementById(`marks-input-${index + 1}`)?.focus();
    }
  }
};
```

Each input has `id={`marks-input-${index}`}` for this to work.

5.8 PDF Download Flow

```
1. metaData arrives (from useQuery)
2. useEffect detects metaData.scriptDetails.pdfSasUrl is set AND pdfBlob is null
3. fetch(pdfSasUrl) → gets Blob
4. setPdfBlob(blob)
5. PDFCanvas renders only when: !isPdfLoading && !pdfError && pdfBlob
```

The Save and Submit buttons are also disabled while `isPdfLoading` is true.

5.9 Save vs Submit

Action	`status` sent	Navigation	Validation
Save Progress	`"in_progress"`	Stay on page	None
Submit Final Evaluation	`"completed"`	`navigate(-1)` after success	All questions must have marks (if pallet exists)

Submit validation:

```
const incompleteQuestions = scores.filter(
  (s) => s.marksAwarded === "" || s.marksAwarded === null || s.marksAwarded === undefined
);
if (incompleteQuestions.length > 0) {
  toast.error("Please enter marks for all questions. Enter 0 if not attempted.");
  return; // block submission
}
```

5.10 Annotation Serialisation

When saving, the canvas annotations are serialised:

```
const annotationData = canvasRef.current?.serializeAnnotations
  ? canvasRef.current.serializeAnnotations()
  : {};

const finalAnnotationData = {
  ...annotationData,           // { annotations: {...} } from canvas
  numberedComments: numberedComments,  // text content of comments from state
};

// Saved as a JSON string:
annotations: JSON.stringify(finalAnnotationData)
```

On load, the saved JSON is parsed and passed to `PDFCanvas` as `initialData`, and `numberedComments` is extracted and set in `OSMGradingPage` state.

5.11 Numbered Comment Synchronisation

Numbered comments have **two parts**:

1. **The annotation box on the canvas** — a `<Rect>` with a number badge, stored in `PDFCanvas` local `annotations` state.
2. **The text content** — stored in `OSMGradingPage` `numberedComments` state as `{ id, number, text }`.

They are linked by `id` (a `Date.now()` timestamp). Operations:

Operation	Canvas	Sidebar
Create	`onAddNumberedComment({ id, number, text: "" })` fired from PDFCanvas	`setNumberedComments(prev => [...prev, comment])`
Edit text	Not affected	`handleNumberedCommentChange(id, newText)` updates text in state
Delete	`canvasRef.current.deleteAnnotationById(id)`	`setNumberedComments(prev => prev.filter(c => c.id !== id))`
Select	`setSelectedCommentId(id)`	Sidebar highlights the matching comment; canvas highlights the matching rect

5.12 Page Jump

```
const handleJumpToPage = () => {
  const pageNum = parseInt(pageInput, 10);
  if (isNaN(pageNum) || pageNum < 1 || pageNum > numPages) {
    toast.error(`Please enter a valid page number between 1 and ${numPages}.`);
    setPageInput(currentPage.toString());
    return;
  }
  canvasRef.current?.scrollToPage(pageNum);
};
```

`scrollToPage` is exposed via `useImperativeHandle` in `PDFCanvas`. It finds the `.pdf-page-wrapper[data-page-number="${pageNumber}"]` element and calls `scrollIntoView`.

The input fires `handleJumpToPage` on both `Enter` key and `onBlur`.

---

6. PDFCanvas — Complete Breakdown

File: `src/components/Canvas/PDFCanvas.jsx`
Type: `forwardRef` component (exposes imperative methods via `useImperativeHandle`)

This is the most complex component in the codebase. It combines PDF rendering with a full-featured drawing system.

6.1 Props

Prop	Type	Description
`pdfFile`	`Blob`	The PDF to render

Prop	Type	Description
`initialData`	`object|null`	Saved annotation data: `{ annotations: { [pageNum]: Annotation[] } }`
`setNumPages`	`function`	Callback: tells parent how many pages the PDF has
`zoom`	`number`	Zoom percentage (50–200). Converted to `scale = zoom / 100`.
`selectedTool`	`string`	Active tool ID: `"select"`, `"pen"`, `"highlighter"`, `"eraser"`, `"tick"`, `"cross"`, `"star"`, `"numbered-comment"`
`currentColor`	`string`	Pen/stamp colour (hex)
`highlighterColor`	`string`	Highlighter colour (hex)
`brushSize`	`number`	Pen stroke width
`onAddNumberedComment`	`function`	`(comment) => void` — fires when a new numbered comment box is drawn
`onNumberedCommentDelete`	`function`	`(id) => void` — fires when a comment annotation is deleted from canvas
`onNumberedCommentChange`	`function`	`(id, newText) => void` — fires when comment text is edited
`numberedComments`	`CommentObject[]`	List from parent state (text content of comments)
`setCurrentPage`	`function`	Reports currently visible page number via IntersectionObserver
`selectedCommentId`	`any`	Currently selected comment ID (for cross-highlighting)
`setSelectedCommentId`	`function`	Called when user clicks a comment on canvas

6.2 Local State

State	Type	Purpose
`numPages`	`number|null`	Total pages (internal copy, also propagated via `setNumPages` prop)
`basePageDimensions`	`{ width, height }`	PDF page size at `scale: 1.0` — used for coordinate normalisation
`annotations`	`{ [pageNum]: Annotation[] }`	All drawn annotations, keyed by page number (1-indexed)
`isDrawing`	`boolean`	Whether a pen/highlighter stroke is currently in progress
`currentDrawingId`	`number|null`	ID of the annotation currently being drawn
`isCreatingComment`	`boolean`	Whether a numbered-comment drag rectangle is in progress
`commentDragStart`	`{ x, y }|null`	Start point of the comment drag
`tempCommentRect`	`{ x, y, width, height }|null`	Live preview rect while dragging to create a comment
`history`	`HistoryState[]`	Undo history stack. Each entry: `{ annotations: {...} }`
`currentStep`	`number`	Current position in the history stack (-1 = empty)
`selectedAnnotation`	`Annotation|null`	The currently selected annotation (for deletion)
`showCommentInput`	`{ id, position }|null`	Controls the floating text input shown after creating a comment
`tempCommentText`	`string`	Text being typed in the comment popup
`isSelectScrolling`	`boolean`	Touch gesture: user is scrolling while in select tool
`isAnnotationToolScrolling`	`boolean`	Touch gesture: user double-tapped to enter scroll mode with annotation tool

State	Type	Purpose
`lastTap`	`number`	Timestamp of last touch — for double-tap detection
`lastTouchY`	`ref`	Last touch Y position — for delta scroll calculation

6.3 Refs

Ref	Purpose
`scrollContainerRef`	Points to the outer scrollable div. Used for IntersectionObserver, scroll manipulation, and bounding rect calculations.
`commentInputRef`	Points to the floating textarea. Auto-focused when `showCommentInput` is set.
`isInitialDataLoaded`	Boolean flag. Prevents `initialData` useEffect from re-running after initial load (so ongoing annotations aren't reset if parent re-renders).

6.4 Exposed Imperative API (`useImperativeHandle`)

The parent (`OSMGradingPage`) holds a `canvasRef` and calls these methods:

```
canvasRef.current.undo()
// Steps back one history entry. Restores annotations from history[currentStep - 1].

canvasRef.current.redo()
// Steps forward one history entry.

canvasRef.current.deleteSelected()
// Deletes whichever annotation is currently selected (if any).

canvasRef.current.deleteAnnotationById(id)
// Searches all pages' annotation arrays and removes the annotation with matching id.
// Used when a comment is deleted from the sidebar.

canvasRef.current.serializeAnnotations()
// Returns: { annotations, numberedComments: [...without text...] }
// Used before saving progress. Note: text content is NOT serialised here —
// it lives in OSMGradingPage state and is merged by the parent.

canvasRef.current.scrollToPage(pageNumber)
// Scrolls the scroll container to bring page N into view.
// Finds .pdf-page-wrapper[data-page-number="${pageNumber}"] and calls scrollIntoView.
```

6.5 Annotation Object Shapes

Every annotation stored in the `annotations` state object is stored in **normalised form** (coordinates are 0–1 relative to base page dimensions). They are denormalised to screen coordinates only at render time.

```
// PEN and HIGHLIGHTER
{
  id: number,           // Date.now()
  type: "pen" | "highlighter",
  points: number[],     // [x0, y0, x1, y1, ...] — NORMALISED (0–1)
  color: string,        // hex colour
  strokeWidth: number,  // NORMALISED (stored at base scale, denormalised on render)
  opacity: number,      // 0.5 for highlighter, 1.0 for pen
}

// TICK, CROSS, STAR (stamps)
{
  id: number,
  type: "tick" | "cross" | "star",
  x: number,            // NORMALISED
  y: number,            // NORMALISED
  text: "✓" | "×" | "★",
  color: "#4CAF50" | "#F44336" | "#FFC107",
  fontSize: number,     // NORMALISED (stored as 24px at base, scaled on render)
}

// NUMBERED_COMMENT
{
  id: number,
  type: "numbered-comment",
  x: number,            // NORMALISED (top-left of rect)
  y: number,            // NORMALISED
  width: number,        // NORMALISED
  height: number,       // NORMALISED
  number: number,       // sequential number (1, 2, 3, ...)
}
```

6.6 Coordinate Normalisation System

This is the most critical technical detail in the entire codebase.

The problem: PDF pages can be zoomed (scale changes). If you store pixel coordinates at zoom 100%, they will be in the wrong position at zoom 150%. The annotation must always align with the same point on the PDF content regardless of zoom level.

The solution: All coordinates are stored as **fractions of the base page dimensions** (0.0 to 1.0). They are converted to screen pixels only at render time by multiplying by the current scaled dimensions.

```
basePageDimensions = { width: W, height: H }  (at scale 1.0, from react-pdf viewport)
scale = zoom / 100

// NORMALISE (screen → storage):
normalised_x = screen_x / (W * scale)
normalised_y = screen_y / (H * scale)

// DENORMALISE (storage → screen):
screen_x = normalised_x * W * scale
screen_y = normalised_y * H * scale
```

Functions:

```
screenToNormalized(screenCoord, baseDimension) {
  return screenCoord / baseDimension;
  // baseDimension here is already W * scale (scaled width)
}

normalizedToScreen(normalizedCoord, baseDimension, currentScale) {
  return normalizedCoord * baseDimension * currentScale;
}
```

Stroke width normalisation: Stroke width is also normalised — stored as the raw `brushSize` value (at scale 1.0 equivalent). On render:

```
denormalized.strokeWidth = (annotation.strokeWidth || brushSize) * scale;
```

This ensures strokes appear the same visual thickness regardless of zoom.

What `basePageDimensions` is: After the first page renders, `onPageRenderSuccess` fires:

```
const onPageRenderSuccess = (page) => {
  if (basePageDimensions.width === 0) {
    const viewport = page.getViewport({ scale: 1.0 });
    setBasePageDimensions({ width: viewport.width, height: viewport.height });
  }
};
```

This captures the PDF's "natural" width and height in points (typically 595×842 for A4). This is set once and never changes. It is the reference for all coordinate maths.

6.7 Tool-Specific Drawing Logic

Select tool

- No drawing occurs.
- Clicking on an annotation calls `handleShapeClick` → sets `selectedAnnotation`.
- Clicking on the transparent `<Rect>` (background) clears selection.
- On mobile: touching the transparent rect enters scroll mode.

Pen / Highlighter

```
mousedown:
  → create new annotation with id = Date.now()
  → push first point (normalised) to annotation.points
  → setIsDrawing(true), setCurrentDrawingId(newId)

mousemove:
  → append normalised point to currentDrawingId annotation's points array
  → setState called on each move (can be many times per second)

mouseup:
  → setIsDrawing(false), setCurrentDrawingId(null)
  → addToHistory(current annotations state)
```

The history is only committed on mouse-up, not during drawing. This prevents the undo history from filling up with every intermediate point.

Eraser

```
mousedown + mousemove:
  → handleErase(pos, pageNumber)
  → Checks each pen/highlighter annotation for proximity to cursor
  → Uses isPointInLine() geometric check
  → If hit: remove that annotation from the array

mouseup:
  → addToHistory (if anything changed)
```

`isPointInLine` algorithm: For each line segment in the stroke's points array, it calculates the perpendicular distance from the cursor to the segment. If that distance is ≤ `(strokeWidth / 2) + 5` pixels, the stroke is considered erased. The `+5` adds a 5px tolerance.

Tick, Cross, Star (stamps)

```
mousedown only:
  → Create annotation with position at click point (normalised)
  → Immediately add to history
  → No drag involved
```

Default colours are hardcoded per stamp type:

- Tick: `#4CAF50` (green)

- Cross: `#F44336` (red)
- Star: `#FFC107` (amber)

These colours are not affected by the colour palette — only pen/highlighter colours use the palette.

Numbered Comment

```
mousedown:
  → setIsCreatingComment(true)
  → Record commentDragStart position

mousemove:
  → Update tempCommentRect (live preview of the rectangle being drawn)
  → Rendered as a dashed purple Rect on canvas

mouseup:
  → If final rect is > 10×10 px:
    → Create normalised annotation
    → Auto-assign next sequential number (max of existing + 1)
    → Call onAddNumberedComment({ id, number, text: "" })
    → Set showCommentInput → floating textarea appears
  → Clean up isCreatingComment, tempCommentRect
```

After the floating textarea submits (`Enter` or `onBlur`):

```
onNumberedCommentChange(showCommentInput.id, tempCommentText)
// → updates the text in OSMGradingPage's numberedComments state
```

6.8 History / Undo System

```
// State:
history = [
  { annotations: { 1: [...], 2: [...] } },  // step 0: initial state
  { annotations: { 1: [..., newAnnotation], 2: [...] } },  // step 1: after first draw
  ...
]
currentStep = index into history array

// addToHistory(newState):
const newHistory = [
  ...history.slice(0, currentStep + 1),  // discard any "future" if we branched
  JSON.parse(JSON.stringify(newState)),   // deep clone to avoid reference mutation
];
setHistory(newHistory);
setCurrentStep(newHistory.length - 1);

// undo():
setAnnotations(history[currentStep - 1].annotations);
setCurrentStep(currentStep - 1);

// redo():
setAnnotations(history[currentStep + 1].annotations);
setCurrentStep(currentStep + 1);
```

Deep clone is essential: `JSON.parse(JSON.stringify(newState))` ensures each history entry is an independent snapshot. Without this, all history entries would reference the same object and undo would have no effect.

History does NOT persist — it lives only in React state. Closing or refreshing the page clears undo history.

6.9 Selection Highlight Rendering

When an annotation is selected (via Select tool), a dashed blue rectangle is rendered around it:

```
if (isSelected && bounds) {
  elements.push(
    <Rect
      stroke="#2563EB"       // blue-600
      strokeWidth={2}
      dash={[5, 5]}          // dashed border
      fill="rgba(37, 99, 235, 0.1)"  // light blue fill
      ...bounds
    />
  );
}
```

`getAnnotationBounds(annotation)` calculates bounding boxes by denormalising coordinates:

- For pen/highlighter: finds min/max of all points, adds 5px padding
- For stamps: centre point ± 15px
- For numbered comments: the rect coordinates ± 2px

6.10 Touch Support

The canvas supports touch devices. A double-tap gesture allows scrolling with annotation tools active (so the evaluator doesn't have to switch to Select to scroll on mobile):

```
Single touch + annotation tool active:
  → Detect tap timestamp
  → If second tap within 300ms: enter isAnnotationToolScrolling mode
  → All subsequent touchmove events scroll the container instead of drawing

Single touch + select tool active:
  → Touching the transparent background rect: enter isSelectScrolling mode
  → All subsequent touchmove events scroll the container
```

Both scroll modes calculate `deltaY` between touch positions and directly set `scrollContainerRef.current.scrollTop`.

6.11 Rendering Logic (`renderAnnotation`)

Called for each annotation in `annotations[pageNumber]`:

```
renderAnnotation(item):
  1. Denormalise coordinates from storage to screen pixels
  2. If item is selected: compute bounds, push a selection <Rect>
  3. Switch on item.type:
     - pen/highlighter → <Line> with denormalised points
     - tick/cross/star  → <Text> with unicode character
     - numbered-comment → <Rect> (box) + <Label>/<Tag>/<Text> (badge) + optional text preview
  4. Return array of Konva elements
```

Numbered comment rendering detail:

```
// Purple box
<Rect fill={isCommentSelected ? "#D1C4E9" : "#EDE9FE"} stroke={...} />

// Number badge (top-left of box)
<Label x={x + 5*scale} y={y + 5*scale}>
  <Tag fill="#8B5CF6" cornerRadius={8} />
  <Text text={String(item.number)} fontSize={10*scale} fill="white" />
</Label>

// Text preview (if comment has text — truncated to 50 chars)
<Text text={comment.text.substring(0, 50) + "..."} fontSize={12*scale} />
```

All font sizes and offsets scale with the zoom factor.

---

7. GradingToolbar — Complete Breakdown

File: `src/pages/OSM/GradingToolbar.jsx`
Role: Left sidebar (140px wide) with tool buttons and action buttons.

7.1 Props

Prop	Type	Description
`selectedTool`	`string`	Currently active tool ID (for highlighting the active button)
`onToolSelect`	`function`	`(toolId) => void` — sets active tool in parent
`onAction`	`function`	`(actionId) => void` — triggers undo/redo/zoom/delete in parent
`onTogglePalette`	`function`	`() => void` — toggles the colour palette floater
`activeColor`	`string`	Current colour (hex) — shown as a preview dot on the palette button
`brushSize`	`number`	Current brush size
`onBrushSizeChange`	`function`	`(newSize) => void` — updates brush size

7.2 Tool Sections

The toolbar is divided into 5 sections with horizontal separators:

Draw section (2×2 grid):

Tool ID	Icon	Description
`select`	`MousePointer2`	Select/move mode. No drawing. Enables annotation selection and deletion.
`pen`	`Pen`	Freehand pen. Uses `currentColor` and `brushSize`.
`highlighter`	`Highlighter`	Semi-transparent overlay. Uses `highlighterColor`, fixed 15px width.
`eraser`	`Eraser`	Erases pen/highlighter strokes by proximity check.

Stamps section (2×2 grid, 3 tools + 1 spacer):

Tool ID	Icon	Description
`tick`	`Check`	Green ✓ stamp. Click to place.
`cross`	`X`	Red ✕ stamp. Click to place.
`star`	`Star`	Amber ★ stamp. Click to place.

Comment section (1 tool):

Tool ID	Icon	Description
`numbered-comment`	`Hash`	Drag to draw a numbered comment box on the canvas.

Color section: A single button showing `<Palette>` icon + a coloured dot preview of `activeColor`. Clicking calls `onTogglePalette`.

Brush Size section (only shown when `selectedTool === "pen"`): An `<input type="range">` with min=1, max=20, step=1. Dynamically appears/disappears with the pen tool.

Actions section (2×3 grid, 5 actions + 1 spacer):

Action ID	Icon	Description
`undo`	`Undo`	Step back in history

Action ID	Icon	Description
`redo`	`Redo`	Step forward in history
`zoomIn`	`Plus`	Increase zoom by 10% (max 200%)
`zoomOut`	`Minus`	Decrease zoom by 10% (min 50%)
`delete`	`Trash2`	Delete selected annotation

7.3 ToolButton Component

```
const ToolButton = ({ label, icon: Icon, isSelected, ...props }) => (
  <button
    title={label}                    // tooltip on hover
    className={`p-3 rounded-lg ... ${
      isSelected
        ? "bg-purple-600 text-white"   // active state
        : "bg-gray-200 text-gray-600 hover:bg-gray-300"  // inactive
    }`}
    {...props}
  >
    <Icon className="h-5 w-5" />
  </button>
);
```

Action buttons (undo, redo, etc.) never have `isSelected` because they are momentary actions, not persistent tool selections.

---

8. DraggableColorPalette — Complete Breakdown

File: `src/components/Canvas/DraggableColorPalette.jsx`

Props

Prop	Type	Description
`initialColor`	`string`	Current hex colour (passed from parent state)
`onChange`	`function`	`(hexColor) => void` — called on every colour change
`onClose`	`function`	`() => void` — closes the palette
`title`	`string`	Window title ("Pen Color" or "Highlight Color")

Behaviour

- Rendered as an `absolute` positioned div (floats over the canvas at `top-24 left-24`)
- Draggable via the header bar (`.drag-handle` class)
- `bounds="parent"` keeps it within the `<FullScreen>` container
- `cancel="button"` means clicking the close button doesn't accidentally start a drag
- The `HexColorPicker` fires `onChange` on every real-time change (no "confirm" step)
- A colour preview swatch is shown below the picker

How the parent decides which state to update:

```
// In OSMGradingPage:
const handleChangeColor = (color) => {
  if (selectedTool === "highlighter") {
    setHighlighterColor(color);
  } else {
    setCurrentColor(color);
  }
};

// The palette always shows the correct current colour:
<DraggableColorPalette
  initialColor={selectedTool === "highlighter" ? highlighterColor : currentColor}
  onChange={handleChangeColor}
  title={selectedTool === "highlighter" ? "Highlight Color" : "Pen Color"}
/>
```

---

9. NumberedCommentsSidebar — Complete Breakdown

File: `src/components/Canvas/NumberedCommentsSidebar.jsx`

Props

Prop	Type	Description
`comments`	`CommentObject[]`	Array of `{ id, number, text }` objects
`onCommentChange`	`function`	`(id, newText) => void` — text edit handler
`onCommentDelete`	`function`	`(id) => void` — delete handler
`selectedCommentId`	`any`	ID of the currently highlighted comment
`onCommentSelect`	`function`	`(id) => void` — called when a comment is clicked

Rendering

Each comment renders:

1. A clickable row wrapper — clicking sets `selectedCommentId` in parent (highlights the matching box on canvas)
2. A numbered purple circle badge showing `comment.number`
3. A `<textarea>` for editing the comment text
4. A `<Trash2>` delete button

Cross-highlight synchronisation: When `selectedCommentId === comment.id`:

- Sidebar row: `bg-purple-100 ring-2 ring-purple-400`
- Badge: `bg-purple-600 text-white` (darker)
- Canvas rect: `fill="#D1C4E9" stroke="#673AB7" shadowBlur=10`

`e.stopPropagation()` on the delete button prevents the row's `onClick` from firing when deleting.

Empty state: When `comments.length === 0`, a grey italic placeholder message is shown: "No comments yet. Use the '#' tool on the canvas to add one."

---

10. DraggablePdfViewer — Complete Breakdown

File: `src/components/Canvas/DraggablePdfViewer.jsx`

Used to show the question paper blueprint alongside the student script without navigating away.

Props

Prop	Type	Description

Prop	Type	Description
`pdfUrl`	`string`	Azure SAS URL to the blueprint PDF
`title`	`string`	Window title shown in header
`onClose`	`function`	`() => void` — closes the viewer

Local State

State	Purpose
`isMinimized`	Toggle between showing/hiding the PDF body (only title bar remains)
`numPages`	Total pages in the blueprint
`currentPage`	Currently shown page (1-indexed)
`scale`	Zoom factor (0.5–3.0, step 0.2)

Layout

```
┌──────────────────────────────────────────────────┐
│  drag-handle title  | ZoomOut | ZoomIn | Min | Close | ← header
├──────────────────────────────────────────────────┤
│                                                  │
│        <Document file={pdfUrl}>                  | ← PDF content
│          <Page pageNumber={currentPage} />       |   (hidden if minimised)
│                                                  │
├──────────────────────────────────────────────────┤
│   [Prev]    Page 2 of 8    [Next]                | ← footer (hidden if minimised)
└──────────────────────────────────────────────────┘
```

- `width={380 * scale}` — the PDF is rendered at a fixed 380px base width that scales with zoom. (This is different from `PDFCanvas` which uses the `scale` prop of `<Page>` — both achieve zoom, just via different props.)
- `isMinimized` conditionally adds `"hidden"` class to both the PDF body div and the footer
- Drag `bounds="parent"` keeps it within the `<FullScreen>` container
- Positioned `absolute top-20 right-20` (starts near the top-right)
- `cancel="button"` prevents drag when clicking zoom/nav buttons

Worker setup (different from PDFCanvas)

```
// DraggablePdfViewer.jsx — uses local bundled worker:
pdfjs.GlobalWorkerOptions.workerSrc = new URL(
  "pdfjs-dist/build/pdf.worker.min.mjs",
  import.meta.url
).toString();

// PDFCanvas.jsx — uses CDN worker:
pdfjs.GlobalWorkerOptions.workerSrc = `//unpkg.com/pdfjs-dist@${pdfjs.version}/build/pdf.worker.min.mjs`;
```

Both approaches work. The CDN approach requires internet access at runtime; the bundled approach works offline (Vite bundles it). This inconsistency is a minor technical debt — both should ideally use the same strategy.

---

11. AnnotationContextMenu — Status

File: `src/components/Canvas/AnnotationContextMenu.jsx`
Status: NOT USED IN ANY ACTIVE FLOW

The component exists and is correctly implemented — it renders a small floating menu with Edit and Delete buttons when positioned at `{ top, left }`.

However, `PDFCanvas.jsx` **never imports or renders it**. There is no `onContextMenu` handler on any Konva element. The component was built as scaffolding for a right-click context menu on annotations but was never connected.

If you want to add right-click-to-delete in the future, you would:

1. Add an `onContextMenu` prop on `<Line>`, `<Text>`, `<Rect>` in `renderAnnotation`
2. Track `contextMenuPosition: { top, left } | null` in state
3. Render `<AnnotationContextMenu>` when `contextMenuPosition !== null`

---

12. Data Flow: Full Lifecycle of a Marking Session

```
1. USER navigates to /osm/grading/SCR-ABC-123?rosterId=R001&flow=osm
   └── OSMGradingPage mounts
   └── Shows fullscreen prompt overlay

2. QUERY fires (useQuery, enabled = user._id + rosterId both present):
   POST /api/osm/evaluation/script/fetch-grading-data/SCR-ABC-123
   Body: { rosterId: "R001", evaluatorId: "user._id" }
   Response (metaData):
   {
     rosterId: "R001",
     scriptDetails: {
       scriptId: "...",
       pdfSasUrl: "https://azure.../script.pdf?sas=...",
     },
     markingPallet: {
       questions: [{ questionNo: "Q1", maxMarks: 10, label: "...", _id: "..." }],
       allowNegativeMarking: false,
     },
     savedProgress: {
       marksBreakdown: [...],
       overallComment: "...",
       annotations: "{...JSON...}",
     },
     bluePrintDetails: {
       blueprint_url: "https://azure.../blueprint.pdf?sas=...",
     }
   }

3. useEffect fires on metaData:
   → Hydrate scores[] from markingPallet (priority 1) or savedProgress (priority 2)
   → Set overallComment
   → Parse savedProgress.annotations JSON → set numberedComments[]

4. useEffect fires on metaData (PDF download):
   → fetch(metaData.scriptDetails.pdfSasUrl)
   → setPdfBlob(blob)
   → isPdfLoading = false

5. PDFCanvas renders:
   → <Document file={pdfBlob}> → react-pdf parses PDF
   → onDocumentLoadSuccess → setNumPages(n)
   → For each page: <Page> renders PDF raster
   → onPageRenderSuccess → captures basePageDimensions (once)
   → <Stage> overlaid on each page
   → initialData passed → annotations hydrated from savedProgress.annotations JSON

6. IntersectionObserver watches .pdf-page-wrapper elements:
   → Updates currentPage as user scrolls

7. USER draws on canvas:
   → GradingToolbar sends tool changes to OSMGradingPage state
   → OSMGradingPage passes selectedTool to PDFCanvas
   → PDFCanvas handles mouse/touch events
   → Annotations stored in local state (normalised coordinates)
   → History updated on mouseup

8. USER clicks "Save Progress":
```

```
   → handleSaveProgress("in_progress")
   → canvasRef.current.serializeAnnotations() → { annotations, numberedComments }
   → Merged with numberedComments text from OSMGradingPage state
   → saveMutation.mutate({ ..., status: "in_progress" })
   → POST /api/osm/evaluation/script/save-progress
   → Toast: "Progress saved successfully!"
   → queryClient.invalidateQueries(["gradingData", ...])
   → Stay on page

9. USER clicks "Submit Final Evaluation":
   → Validate: all scores have marksAwarded (if pallet exists)
   → handleSaveProgress("completed")
   → Same save flow
   → Toast: "Evaluation submitted successfully!"
   → navigate(-1) → back to evaluator dashboard
```

---

13. Annotation Coordinate System (Critical)

Understanding this prevents the most common bug: annotations appearing in the wrong position after zooming or on different screen sizes.

The Rule

Never store screen pixels. Always store normalised values (0.0–1.0).

```
          Screen pixel coordinates
          ┌──────────────────────────────┐
          │ PDF Page (rendered at zoom 150%) │
          │ width = 595 * 1.5 = 892px    │
          │                              │
          │    Draw a tick at (300, 200)  │
          │                              │
          └──────────────────────────────┘
                    ↓ normalise
          normalised_x = 300 / 892 = 0.336
          normalised_y = 200 / (842 * 1.5) = 0.158
                    ↓ store in state
          { type: "tick", x: 0.336, y: 0.158, ... }

                    ↓ at zoom 100%
          screen_x = 0.336 * 595 * 1.0 = 200px
          screen_y = 0.158 * 842 * 1.0 = 133px
          → Tick renders at proportionally correct position ✓
```

Where Normalisation Happens

- **mousedown (pen/highlighter):** `normalizeAnnotation()` called on new annotation before storing
- **mousemove (pen/highlighter):** Each new point normalised via `screenToNormalized()` before appending
- **mousedown (stamps):** `normalizeAnnotation()` called on new annotation
- **mouseup (numbered-comment):** `normalizeAnnotation()` called on the final rect
- **render:** `denormalizeAnnotation()` called on every annotation before passing to Konva components

Why `baseDimension` in `screenToNormalized` is `W * scale`, not just `W`

The function signature is:

```
screenToNormalized(screenCoord, baseDimension)
// returns: screenCoord / baseDimension
```

When called:

```
screenToNormalized(pos.x, basePageDimensions.width * scale)
// = pos.x / (W * scale)
// This gives a value in [0, 1] relative to the *currently rendered* page width
// Which is the same as the actual fraction of the page, at any zoom level
```

And when denormalising:

```
normalizedToScreen(normalizedCoord, baseDimension, currentScale)
// = normalizedCoord * baseDimension * currentScale
// = normalizedCoord * W * scale
// = the correct screen pixel at the current zoom
```

---

14. Dual-Flow Architecture (OSM vs NU)

The grading page supports two backend flows selected by the `?flow=` query parameter:

```
?flow=osm  (default)
  Fetch: POST /api/osm/evaluation/script/fetch-grading-data/:code
  Save:  POST /api/osm/evaluation/script/save-progress
  DB: Standard OSM collections (Scanned_Scripts, Evaluation_Rosters, etc.)

?flow=nu
  Fetch: POST /api/cbt/offline/evaluation/script/fetch-grading-data/:code
  Save:  POST /api/cbt/offline/evaluation/script/save-progress
  DB: NU-specific offline CBT collections (Scanned_Scripts_NU, etc.)
```

The component API/UI is **completely identical** between the two flows. The only difference is which HTTP endpoint is called. This design means new backend flows can be added without any UI changes — just add a new flow ID and API function.

The `isNuFlow` boolean is derived from the URL and is part of the React Query cache key:

```
queryKey: ["gradingData", uniqueScriptCode, rosterId, isNuFlow]
```

This ensures that if a user somehow loads the same script URL with different `?flow=` values, they get separate cache entries.

---

15. State Management Summary

State Location	What It Holds	Scope
`OSMGradingPage` local state	scores, comments text, zoom, tool selection, colours, brush size, fullscreen flags, pdfBlob	Per-session, lost on unmount
`PDFCanvas` local state	All drawn annotations (object keyed by page), history stack, drawing in-progress flags, base page dimensions	Per-session, lost on unmount
Redux `auth` slice	`user._id` (evaluator ID)	Persisted in localStorage
React Query cache	`metaData` (from server: script details, pallet, saved progress)	Cached per query key, invalidated on save
`localStorage["token"]`	JWT token	Cross-session persistence

Nothing in this marking flow uses Redux for annotation state. The `annotationsSlice` in Redux was built for a different, older grading flow. Do not use it here.

---

16. Keyboard & Touch Interactions

Interaction	Action

Interaction	Action
`Ctrl + ArrowDown`	Move focus to next marks input
`Ctrl + ArrowUp`	Move focus to previous marks input
`Enter` (in page jump input)	Jump to entered page number
`Blur` (leaving page jump input)	Same as Enter
`scroll wheel on marks input`	Blur the input (prevents accidental value change)
`Enter` (in comment popup textarea)	Save comment text
`Shift+Enter` (in comment popup)	Insert newline (not submit)
`Escape` (in comment popup)	Cancel comment (discard typed text)
Touch: double-tap on canvas	Enter scroll mode (annotation tool stays active, touch moves scroll instead of draw)
Touch: single touch on transparent Rect (select mode)	Enter scroll mode
ESC / F11	Exit fullscreen → re-show fullscreen prompt banner

---

17. Known Gaps & TODOs in the Code

Issue	Location	Detail
`AnnotationContextMenu` not wired up	`PDFCanvas.jsx`	Component exists but is never rendered. Right-click context menu on annotations is not implemented.
Stats in `EvaluationDashboard` are hardcoded	`pages/EvaluationFlow/Dashboard.jsx`	`totalScripts: 2847`, `evaluatedToday: 156` etc. are static values — needs a real API call.
Inconsistent pdf.js worker strategy	`PDFCanvas.jsx` vs `DraggablePdfViewer.jsx`	One uses CDN, the other uses bundled worker. Should be unified.
Commented-out `evaluatorId` in `rosterSlice.js`	`store/slices/rosterSlice.js:33`	A hardcoded evaluator ID is commented out. Cleanup needed.
`handleAction("help")` is a no-op	`OSMGradingPage.jsx:414`	The help action has a comment but no implementation.
Annotations not in Redux	`OSMGradingPage.jsx`	All annotation state is local to `PDFCanvas`. If the evaluator accidentally refreshes mid-session before saving, all unsaved annotations are lost. Auto-save could help here.
No response interceptor on `api.js`	`src/api.js`	If the server returns 401 (expired token), the error is swallowed silently. A 401 interceptor that dispatches `logout()` would improve UX.

Last updated: May 2026. For questions, check with the frontend lead or raise a GitHub issue.